

Memoria Práctica 3 — Q-learning para seguimiento de línea

1. Introducción

Controlador en Python para el Khepera IV en Webots. El robot aprende a seguir una línea negra mediante **Q-learning tabular** sin modelo del entorno. Un módulo reactivo independiente esquiva los obstáculos sin formar parte del aprendizaje.

Código: `controllers/p3_controller/p3_controller.py`.

2. Estados

El espacio se reduce a **tres estados discretos** calculados en `get_state` a partir de los 4 IR de suelo (lateral izq., central izq., central der., lateral der.) y los dos umbrales del enunciado: $> 750 = \text{blanco}$, $< 500 = \text{negro}$.

Estado	Significado	Condición
S1	Línea perdida por la izquierda	$\text{central_izq} > 750 \text{ Y } \text{lateral_der} < 500$
S2	Línea perdida por la derecha	$\text{central_der} > 750 \text{ Y } \text{lateral_izq} < 500$
S3	Resto (sobre línea, o totalmente fuera)	cualquier otra combinación

Mantener sólo tres estados deja una Q-table 3×3 , lo que favorece una convergencia rápida.

3. Acciones

Tres acciones en `run_action`: girar derecha (A1), girar izquierda (A2) y avanzar recto (A3). Dos decisiones de diseño marcan el comportamiento:

Giros como arcos dinámicos, no como pivotes. Las primeras pruebas con giros estáticos (una rueda hacia adelante y la otra hacia atrás) permitían al robot quedarse en el sitio alternando estados y generando un bucle de recompensa que no nos acercaba al comportamiento deseado. Al añadir un parámetro para controlar la velocidad de la rueda

interna (`CURVE_INNER = 1 rad/s` frente a `CRUISE_SPEED = 15 rad/s` en la exterior), creamos un movimiento de curva en lugar de un giro en el sitio.

Duración asimétrica. `ACTION_STEPS_FWD = 1` tick para avanzar y `ACTION_STEPS_TURN = 2` ticks para girar. La acción de giro dura más tiempo de simulación que el avance, favoreciendo correcciones rápidas cuando el robot se desvía.

4. Recompensa

El enunciado prohíbe codificar la recompensa a priori, por lo que `compute_reward` la deriva de la experiencia comparando para cada sensor su lectura antes y después de la acción. Con `w = REWARD_WEIGHTS[i]`:

Transición del sensor	Contribución
blanco → negro (gana línea)	$+0.5 \cdot w$
negro → blanco (pierde línea)	$-0.75 \cdot w$
negro → negro (mantiene línea)	$+1.25 \cdot w$
blanco → blanco (sigue fuera)	$-w$

Los pesos son `[1.0, 2.0, 2.0, 1.0]`: los centrales (1 y 2) pesan el doble, dando prioridad a estar centrado sobre la línea frente a sólo rozarla con un lateral.

Bonus de configuración completa. Tras la suma por sensor se evalúa el estado final completo:

- Los **4** sensores en negro: **+BONUS_FULL** (totalmente centrado o sobre un cruce).
- Los **4** sensores en blanco: **-BONUS_FULL** (totalmente fuera de la línea).

Con `BONUS_FULL = 1.0`, este término amplifica las señales más informativas y permite a la Q-table distinguir, dentro de S3, entre "centrado" y "fuera del todo".

5. Actualización de la Q-table

Regla no-determinista indicada en el enunciado, implementada literalmente en `update_q`:

$$Q_n(s,a) \leftarrow (1 - \alpha_n) \cdot Q_{n-1}(s,a) + \alpha_n \cdot [r + \gamma \cdot \max_{a'} Q_{n-1}(s',a')]$$

Parámetro	Valor	Significado
α	<code>1 / (1 + visits[s,a])</code>	Tasa adaptativa; decrece con la experiencia y estabiliza
γ	<code>0.5</code>	Factor de descuento entre recompensa inmediata y valor futuro
r	salida de <code>compute_reward</code>	Refuerzo escalar observado
s'	<code>get_state(curr_g)</code>	Estado tras la acción

La Q-table (`float64`) y el contador de visitas (`int64`) son matrices 3×3 inicializadas a ceros en cada arranque. No hay persistencia entre ejecuciones: simplifica el ciclo de prueba y permite comparar políticas en condiciones idénticas.

6. Exploración vs. explotación

Política ϵ -greedy en `select_action`:

- Con probabilidad ϵ : acción aleatoria uniforme.
- Con probabilidad $1 - \epsilon$: `argmax_a Q[s, a]`.

`epsilon` hace decaer ϵ **linealmente** de 1 a 0 a lo largo de `EPS_DECAY_ITERS = 500` iteraciones. El agente empieza explorando puramente y, hacia el final del entrenamiento, explota la política aprendida.

7. Evitación de obstáculos

No se aprende: es un módulo reactivo (`avoid_obstacles`) sobre los 3 IR de proximidad que miran hacia adelante (FL, F, FR). Se usan **umbrales asimétricos**:

Sensor	Umbral	Justificación
--------	--------	---------------

Central (F)	<code>OBSTACLE_THR_CENTER = 250</code>	Bajo → detecta a distancia. Lo que está justo enfrente siempre es amenaza.
Laterales (FL, FR)	<code>OBSTACLE_THR_SIDE = 500</code>	Alto → sólo cuasi-contacto. Una pared vista en paralelo no es colisión.

Cuando dispara la maniobra:

1. **Sentido del giro.** Si $FL \geq FR$, el obstáculo está más a la izquierda → giro a la derecha. En otro caso, a la izquierda. Siempre se gira hacia el lado más despejado.
2. **Ángulo según fuente del disparo.** Central → **45°** (corrección amplia). Lateral → **10°** (corrección mínima). Si ambos disparan, prevalece el central.
3. **Ejecución exacta.** `turn_in_place` mide la rotación con los encoders de las ruedas: para un giro in-place simétrico cada rueda recorre $\Delta\phi = \theta \cdot (WHEELBASE/2) / WHEEL_RADIUS$. La velocidad `AVOID_TURN_SPEED = 25 rad/s` sólo influye en cuánto tarda, no en el ángulo final.

Tras la maniobra se releen los sensores de suelo y el bucle Q-learning retoma el control desde el nuevo estado.

8. Bucle principal

1. Inicializar motores (modo velocidad), encoders y sensores de suelo/proximidad. Un primer `step` para que las lecturas estén disponibles.
2. Inicializar `Q` y `visits` a ceros.
3. En cada iteración:
 1. `avoid_obstacles()` — si hay obstáculo, ejecuta el giro y reinicia el estado.
 2. Calcular ϵ y elegir la acción (ϵ -greedy).
 3. Leer suelo previo, ejecutar la acción con `run_action`, leer suelo posterior.
 4. Calcular recompensa (`compute_reward`) y estado siguiente (`get_state`).
 5. Actualizar la Q-table (`update_q`).
 6. Log e incremento del contador.

9. Conclusiones

El controlador cumple los requisitos del enunciado: aprendizaje tabular con recompensa derivada de la experiencia, regla de actualización no-determinista con tasa adaptativa y transición progresiva de exploración a explotación. Las decisiones más relevantes para el comportamiento final fueron:

- **Sustituir giros estáticos por arcos dinámicos**, para eliminar bucles parasitarios sin desplazamiento real.
- **Duraciones asimétricas** entre avance y giro, que hacen del recto la opción más rentable en S3.
- **Bonus de configuración completa** sobre los 4 sensores, para amplificar las señales más limpias y desambiguar S3.
- **Umbrales y ángulos diferenciados** en la evitación (central anticipado a 180° , lateral inminente a 60°), que reducen falsos positivos al avanzar paralelo a paredes y dimensionan la respuesta al grado real de amenaza.

Una pequeña anotación que nos gustaría realizar es que a la hora de desarrollar el algoritmo empleamos velocidades bajas de $\sim 4\text{rad/segundo}$. Durante una corrección en clase se destacó que el robot se movía muy lento.

Al subir la velocidad el comportamiento del robot empeoró considerablemente y tuvimos que pasar un tiempo ajustando parámetros tanto de movimiento como de recompensa hasta que finalmente conseguimos un comportamiento estable a una velocidad de 15 rad/s . No obstante sigue siendo menos estable que la versión anterior y hay que ejecutar la simulación una o dos veces hasta dar con una con el aprendizaje correcto.